

光稳智检——面向复杂光照环境的工业视觉异常检测与缺陷定位平台

应用方案书

一、项目背景与意义

1.1 行业背景

工业产品表面异常检测是质量控制中的重要环节。以正常样本建立参考、再识别偏离正常模式的图像，有助于缓解缺陷样本不足的问题。然而，实验室数据与现场环境的光照差异可能使阴影、反光被误当成缺陷。本项目围绕这一具体问题，开发可演示的软件原型，并用公开数据集进行初步验证。

1.2 现有问题

当前工业视觉检测面临以下挑战：

- 光照环境不稳定**：生产车间的光照条件复杂多变
- 缺陷样本稀缺**：工业生产追求“零缺陷”，缺陷样本极为稀少
- 跨场景验证不足**：仅在固定光照数据集上得到高分，不代表能在新光照或真实生产线可靠运行。

二、行业痛点分析

2.1 光照变化导致的误报

工业生产环境中，光照条件复杂多变，导致检测系统产生大量误报。

2.2 核心挑战

挑战	描述	影响
阴影误判	正常产品在阴影区域被误判	误报率上升
反光干扰	金属表面反光被误判为异常	检测失效
亮度波动	检测结果因光照变化不稳定	结果不可靠

2.3 需求分析

官方要求说明目标用户群体与功能需求，本节按此逐项列出。以下需求来自团队对教学实验室与中小制造企业的走访与问卷反馈，**属定性调研结论，尚未做规模化定量统计。**

2.3.1 目标用户群体

用户群体	典型场景	核心诉求	对成本/部署的约束
高校与职校视觉质检教学团队	课程实验、毕业设计、算法对比实验	需要可复现、可解释的异常检测演示平台	无专用服务器，希望单机离线运行
中小企业质检技术人员	小批量、多品种产线抽检	不希望采集大量缺陷样本即可起步	无标注团队，无 GPU 预算
算法研究初学者	复现 PatchCore、做光照鲁棒性对比	需要完整可运行的参考实现与真实数据指标	希望给出失败案例而非只报最好结果
竞赛与项目评审场景	现场演示运行过程	需要真实、连续、可交互的操作画面	需在断网环境可运行

2.3.2 功能需求

编号	需求	优先级	当前实现状态
F1	选择产品类别并加载对应正常特征记忆库	必须	已实现（15 类）
F2	上传单张图像并输出正常/异常判定、异常分数与阈值	必须	已实现
F3	输出像素级异常热力图与叠加图，提示可疑位置	必须	已实现（非缺陷概率）
F4	批量检测多张图像并汇总正常/异常数量	必须	已实现
F5	批量结果可逐张切换查看热力图	重要	已实现
F6	检测历史追溯与 CSV 报告导出	重要	已实现
F7	展示基线与改进模型的离线对比结果	重要	已实现（只读记录，非实时切换）
F8	阈值可现场调整以观察误报与召回的权衡	次要	已实现（仅当前运行生效）
F9	自动阈值校准（按目标误报率）	期望	未实现 ，见技术文档第 2.2 节
F10	企业自有数据训练与多人协同	期望	未实现 ，需现场数据与授权

2.3.3 非功能需求

维度	要求	当前达成情况
离线可用	演示现场不依赖网络	已达成：骨干权重与记忆库随包内置，断网可运行
启动成本	无需专用硬件	已达成：CPU 推理，常驻内存约 1.1 GB
响应速度	单张秒级出结果	已达成：实测约 0.13–0.33 秒/张（CPU）
隐私与合规	不上传用户图像到外部服务	已达成：全部本地处理，不发起外部请求
可解释性	结果须可追溯、不误导	已达成：热力图明确标注非缺陷概率；误报率与召回率成对披露
误报控制	控制正常图误报	未达成 ：当前 34.27% 误报率偏高，定位为高召回预筛工作点

三、系统总体方案

3.1 开发工具与运行环境

官方要求介绍所用开发工具，本节列明全部工具链。**所有依赖均已随包提供或可通过 requirements.txt 一次安装，运行阶段不需要联网。**

类别	工具 / 组件	版本要求	用途
编程语言	Python	3.10 – 3.13（已在 3.11.9 与 3.13.9 实测）	全部后端与算法代码
深度学习框架	PyTorch	≥ 2.0	特征提取、距离计算、张量运算
预训练模型库	TorchVision	≥ 0.15	WideResNet-50-2 网络结构与 IMAGENET1K_V2 权重
Web 框架	Flask	≥ 2.3	后端接口与页面服务
WSGI 工具	Werkzeug	≥ 2.3	文件上传安全处理（secure_filename）、静态文件服务
图像处理	Pillow、OpenCV	Pillow ≥ 10.0、opencv-python ≥ 4.8	图像读写、预处理、热力图配色（COLORMAP_JET）
数值计算	NumPy	≥ 1.24	记忆库采样与指标计算
前端	HTML5 + CSS3 + 原生 JavaScript	无额外依赖	交互界面，不使用前端框架以便离线与易读
操作系统	Windows 10 / 11	—	演示与评测环境
界面图标	Font Awesome	—	界面图标
自动化测试工具	Playwright	—	界面走查与截图复现
文档生成工具	reportlab、python-pptx	—	本方案书 PDF 与答辩 PPT 的生成脚本

说明：一键启动脚本 光稳智检_iCAN演示启动.bat 会自动检查 Python 与依赖是否就绪；start.bat 用于首次安装依赖。依赖清单见 01_可运行系统/requirements.txt，与代码实际 import 一一对应。

3.2 技术架构

系统采用“正常样本特征提取—记忆库构建—异常评分”的三阶段架构。PatchCore 是已有公开方法，本项目在其基础上研究光照增强、阈值校准与交互式展示：

- 特征提取：WideResNet-50骨干网络
- 记忆库构建：基线为 Coreset 贪心采样；**本项目改进模型改用固定容量的均匀随机抽样**（机制见 5.4 节）
- 异常评分：特征与记忆库的最近邻距离计算

系统定位为软件赛道的可运行原型：输入单张或多张产品图像，输出图像级异常判断、异常分数与定位热力图。当前公开演示依赖已打包的15类模型权重，不等同于已经在真实产线完成验证。

3.3 系统功能

功能	说明
产品类别选择	支持15类MVTec工业产品
图像上传	支持拖拽和点击上传
AI检测	自动判断正常/异常
热力图	提示可疑区域（相对异常响应，非缺陷概率）
批量检测	支持多张图片；可视化区可逐张切换查看每张的原图/热力图/叠加图
报告导出	CSV格式检测报告

四、核心AI技术

4.1 PatchCore基线

PatchCore是一种基于正常图像特征记忆库的异常检测方法。对输入图像的局部特征寻找最近邻，以距离构造异常图；图像级分数用于筛查，热力图用于提示可疑位置。多尺度特征与 Coreset 采样属于该基线的既有组成，不能作为本项目独创技术申报。

4.2 光照鲁棒增强

在正常训练图像上设计5类光照增强，以模拟亮度、色温、阴影和空间照度变化。下表为当前方案中的参数范围，仍需用消融实验确认每项增强的实际贡献：

增强类型	参数范围	模拟场景
亮度变换	0.6-1.4	灯光明暗变化
对比度变换	0.7-1.3	光源距离变化
色温变换	±0.15	光源色温差异
阴影遮挡	概率0.4	设备或人员遮挡
不均匀光照	概率0.4	光源分布不均

4.3 类别自适应阈值

针对每个产品类别设置阈值。阈值应仅在训练集划出的正常验证集或另行确定的验证集上校准，不能使用测试集标签反向调参；界面中的手动调阈值用于演示与风险观察，不代表已完成工业场景校准。

五、项目特色与待验证创新

5.1 光照鲁棒的记忆库构建

将多种光照扰动引入正常样本特征建库，并以未知光照测试集检验鲁棒性。这是本项目要验证的核心研究假设，不把局部类别的提升表述为整体已证实的结论。

5.2 场景化评测与可解释呈现

在固定训练/测试划分和相同记忆库容量下，按类别报告图像级、像素级指标及失败案例；界面同时呈现异常分数、阈值、热力图与离线对比结果，方便观察误报来源。多尺度特征提取沿用 PatchCore 基线。

5.3 类别自适应阈值校准

将阈值校准与产品类别绑定，避免用一个全局数值覆盖所有类别；其有效性还需独立验证。

5.4 记忆库构建方式与提速的真实来源

先把机制说清楚，避免被误读为一项无代价的算法创新。

	基线	改进模型
建库所用特征	原始训练图像	原始图像 + 3 个光照增强变体
采样方式	贪心 k-center coresets (逐点选取距当前集合最远的点，需迭代 $n-1$ 次，每次遍历全部特征)	均匀随机抽样 (np.random.choice)
最终记忆库容量	每类相同	每类相同 (与基线一致)

同一轮 bottle 配对记录中，记忆库构建耗时由739.56秒降至7.74秒，约为95.5倍。**这一提速来自把“贪心 coresets 采样”替换为“均匀随机抽样”**：前者复杂度约为 $O(\text{采样点数} \times \text{全部特征数})$ ，后者为 $O(\text{采样点数})$ ，因此耗时下降约两个数量级；两侧的最终记忆库容量完全相同、推理耗时基本相同（比值 ≈ 1.0 ），所以提速并非来自缩小记忆库或加速推理。

关于“提速是否以精度为代价”，需要更正一处早期推断。

早期版本的技术说明曾推断：随机抽样削弱了特征子集的代表性，因而应当对 6 类配对实验中 3.93 个百分点的平均下降负责。对照实验未支持这一推断——在固定类别、固定记忆库容量、仅改变采样方式的条件下，随机抽样在 bottle 上取得 0.9935 ± 0.0014 （5 个种子），贪心 coresets 为 0.5825，即随机抽样并不更差（详见技术文档第 4.2 节）。因此**该 3.93 个百分点不能归因于采样方式的替换**。

同时必须指出一项更根本的可复现性缺陷：三个实验脚本**均未设置随机种子**，而两臂建库都使用含 RandomCrop、随机翻转与 ColorJitter 的训练变换、且只对训练集做单次遍历，每张训练图只贡献一个随机视角。因此**配对实验的基线侧与改进侧都是随机的**，记录的差值中含有未知比例的运行间波动，无法精确复现（详见技术文档第 4.3 节）。要把它做成可信结论，需固定种子并重复多次、报告均值与方差。

该提速数字只对应 bottle 这一类别与这一次运行，不作为所有类别或端到端部署的通用加速结论。

六、现有实验记录与局限

以下数字按团队原始结果 JSON 重新核对。15类汇总、6类配对、MVTec AD 2 属于不同轮次，不混合计算平均值。修订包附有对应结果 JSON，但仍缺完整训练/评测环境记录及逐样本预测文件，尚不能仅凭打包材料独立复现。AUROC 是排序能力指标，不能直接等同于检测准确率或生产线误报率。

6.1 MVTec AD 15类平均性能

指标	数值
平均Image AUROC	88.13%
平均Pixel AUROC	92.43%
平均F1 Score (阈值取优最优值, best-case)	91.21%
平均AUPRO	73.72%
Image AUROC 类别区间	50.38% (screw) ~ 100% (leather) , 15类中12类不低于84%
正常图误报率 (类别均值)	34.27% (中位31.25%; screw 100%、pill 84.62%, bottle与leather为0)
缺陷召回率 (类别均值)	93.49% (最低 transistor 75.0%, 最高 leather 100%)

上表为改进模型在该轮的15类结果；该轮基线只完成了bottle一类，因此本节不给出15类基线对照。表中F1是**在阈值上取最大值的最优F1**（等价于配对记录中的 `image_f1_max`），不是系统当前固定阈值下的F1；把它与Image AUROC并列时容易被误读为同一工作点上的准确率，特此注明。误报率与召回率两行均对应系统当前出厂阈值（与 `model.py` 的 `REAL_THRESHOLDS` 完全一致）：前者是该阈值下被判为异常的正常图比例，后者是被正确判为异常的缺陷图比例。**两者必须成对阅读——只报误报率不报召回率属于选择性披露。**

进一步核对阈值来源后发现一个更根本的问题：随包程序的15个出厂阈值与配对记录中的 `image_threshold_from_test`（在测试集上取最优F1得到的阈值）性质一致，而明显不同于同一记录中的 `calibration_threshold`（六类平均相差5.66个单位，详见技术文档第2.1节）。也就是说，**这些阈值是在测试集上取优得到的，而不是在独立验证集上标定的**。因此上表的F1属于最好情况，上表的误报率就是这个取优工作点上的真实误报。

本轮还用**仅正常样本**做了重标定实验：把每类正常图对半分为标定集与评估集，只用标定集的正常分数分位数定阈值（不接触缺陷标签）。结果是——把误报率从约31%降到约10%，召回率会同期从93.5%降到76.9%，即约23个百分点的缺陷被漏检；**screw** 尤为极端（召回率99.2%→4.2%，因其Image AUROC仅0.5038，正常与缺陷分数几乎重合，调阈值只能翻转失败模式）。**因此本系统保留当前阈值，并将其定位为"高召回预筛工作点"**：系统用于批量预筛加入人工复核，该场景下漏检代价高于误报，偏向高召回是合理取舍。详细权衡表与逐类数据见技术文档第2.2节。

6.2 基线与改进对比（6类配对实验）

类别	基线AUROC	改进AUROC	变化
bottle	94.52%	98.41%	+3.89 pp
hazelnut	99.57%	99.79%	+0.21 pp
cable	92.05%	82.40%	-9.65 pp
capsule	90.47%	83.25%	-7.22 pp
carpet	99.92%	99.12%	-0.80 pp
grid	88.81%	78.78%	-10.03 pp
平均	94.22%	90.29%	-3.93 pp

变化单位为百分点（pp），不是相对百分比。该轮6类中2类提高、4类下降，平均表现低于基线；现阶段不能宣称改进方法在标准数据集上整体优于 PatchCore。变化值由未舍入的原始AUROC相减后保留两位小数（hazelnut实际为+0.214个百分点）。后续应重点分析 cable、capsule、grid 的退化并做分组/消融实验。

6.3 MVTec AD 2光照域偏移实验

类别	基线→改进Image AUROC	变化
can	50.12%→48.46%	-1.67 pp
fabric	46.55%→42.78%	-3.77 pp
fruit_jelly	91.08%→98.25%	+7.17 pp
rice	55.87%→42.43%	-13.44 pp
sheet_metal	60.14%→72.08%	+11.94 pp
vial	66.48%→71.05%	+4.57 pp
wallplugs	37.69%→47.24%	+9.56 pp
walnuts	63.76%→74.26%	+10.50 pp
8类宏平均	58.96%→62.07%	+3.11 pp

这份本地结果中5类提高、3类下降；平均改善与 can、fabric、rice 的明显退化需要同时报告。还有两点必须一并说明：

1. **像素级指标不升反降。** 同一轮8类宏平均中，Image AUROC 提升 3.11 pp，但 Pixel AUROC 由 80.18% 降至 78.24% (-1.94 pp)、AUPRO 由 37.66% 降至 37.35% (-0.31 pp)、Pixel AP 由 11.64% 降至 8.57% (-3.07 pp)、Pixel F1（阈值取优）由 17.37% 降至 13.12% (-4.26 pp)。即改进主要体现为图像级排序能力，缺陷定位反而略有变差。
2. **误报率仍然很高。** 按类别汇总的正常图误报率均值由 88.07% 变为 85.83%，改善后仍有约 85.83% 的正常图在当前阈值下被判为异常（单光照条件下为 1.0%~15.8%，汇总口径下显著更高）。

因此本方法在 MVTec AD 2 上只取得了有限的图像级排序提升，像素级定位与误报率均未改善，**不足以支撑"光照鲁棒性已显著改善"或"已解决光照误报"的结论**。需补充划分、提交记录和官方评测口径后，才可将本地结果表述为正式基准成绩。

6.4 效率提升

指标	数值
记忆库构建提速	bottle单次配对约95.5倍
基线构建时间	739.56秒
改进构建时间	7.74秒
CPU检测时间	当前抽样约0.15秒/张（待多次计时）

七、系统功能

7.1 系统架构

- 前端：HTML5 + CSS3 + JavaScript

- 后端: Flask + PyTorch
- AI模型: PatchCore + 光照鲁棒增强

7.2 核心功能

- 单张图片检测
- 批量检测
- 热力图可视化
- CSV报告导出
- 离线实验对比 (展示预先记录的汇总数据, 非实时切换模型)

系统界面展示

以下截图取自本包可运行程序的真实操作过程：选择 bottle 类别并加载记忆库后，依次上传正常样本与缺陷样本，查看异常分数、热力图与叠加图，随后查看离线配对实验结果并执行一次批量检测。

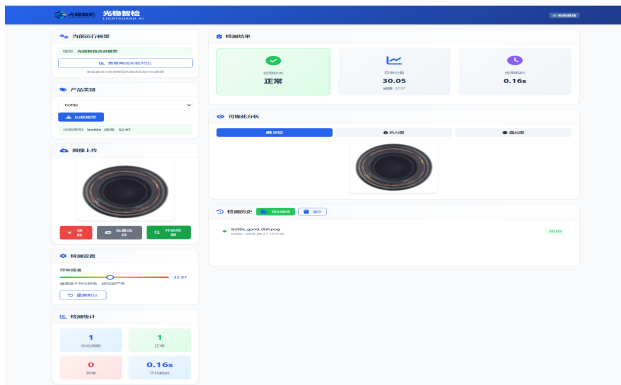


图 1 正常样本：判定「正常」（分数 30.05 < 阈值 32.97）

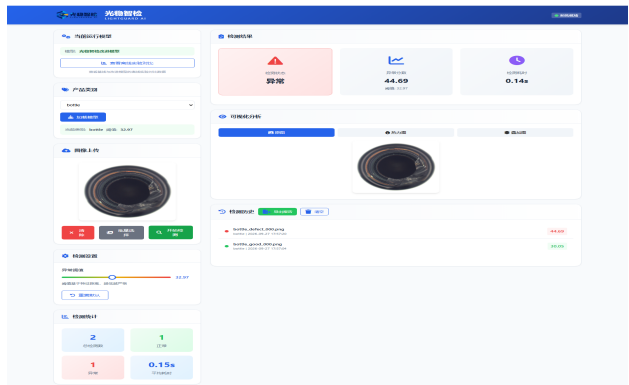


图 2 缺陷样本：判定「异常」（分数 44.69 > 阈值 32.97）

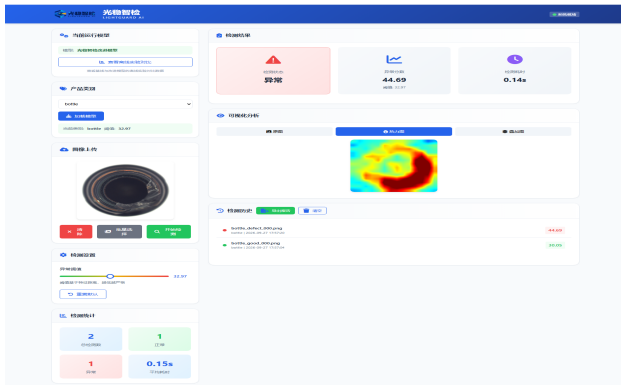


图 3 异常热力图：响应集中在破损位置

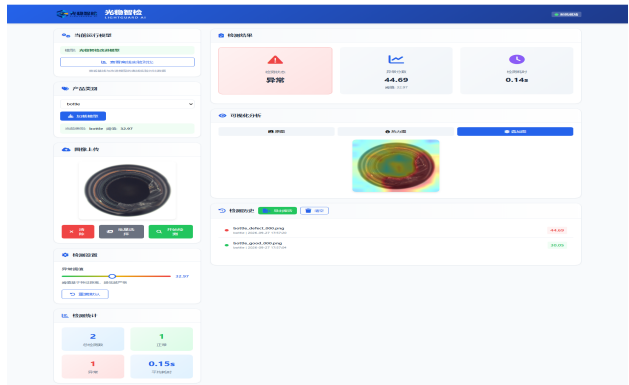


图 4 缺陷叠加图：热力图与原图叠加

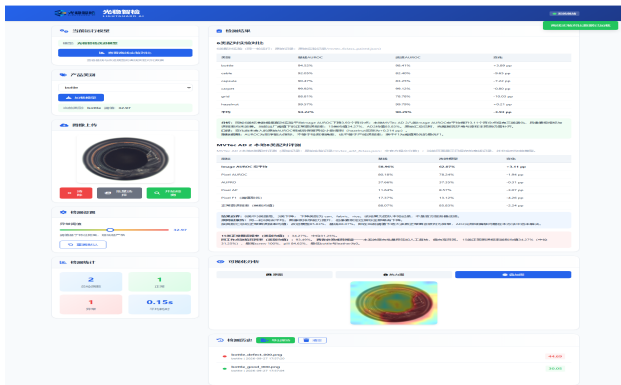


图 5 离线配对对比：6类配对与MVtec AD
2像素级指标、误报率披露

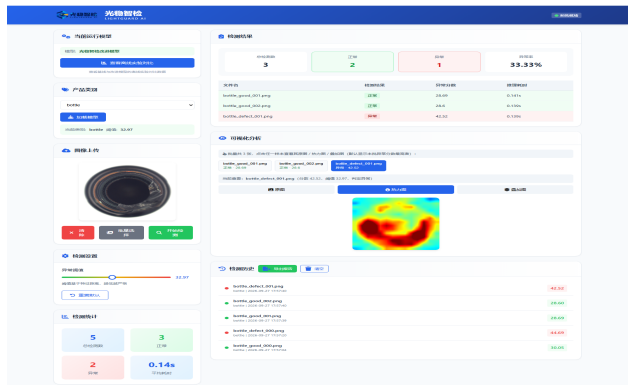


图 6 批量检测：3 张样本，2 正常 1
异常；可视化区可逐张切换查看

7.4 使用说明、操作流程与交互指南

官方要求提供完整使用说明，本节按"启动 → 检测 → 查看结果 → 导出"给出操作流程与界面交互说明。

7.4.1 如何取得并运行本系统（与"可运行程序"栏对应）

官方要求"可运行程序"提供可运行的链接或应用程序源代码。本项目按"提供源代码"的方式交付，代码仓库为：

<https://github.com/SHY10086332/lightguard-ai>

该仓库同时包含两部分：**项目展示站**（GitHub Pages，展示介绍与截图）与**可运行程序源代码**（Flask 后端、推理代码、网页界面）。模型权重因体积超出 GitHub 单文件上限（骨干权重 263 MB），作为该仓库 **Releases** 的附件提供。

与演示视频的关系：演示视频中展示的是本系统在**本地真实运行**的画面（浏览器地址为 <http://127.0.0.1:5000>）。原因是本系统的异常检测依赖 PyTorch 推理与本地记忆库，**无法部署为静态网页**，按设计在本地离线运行。评审按下列步骤即可复现视频中的同一界面与同一结果。

取得与运行（Windows，约 5 分钟，首次需联网装依赖）：

- 1. 克隆仓库：`git clone https://github.com/SHY10086332/lightguard-ai.git`
- 2. 在该仓库 **Releases** 页面下载 `lightguard-weights.zip`，解压后把 `pretrained/` 与 `weights/` 两个目录放入 `01_可运行系统/`。
- 3. 安装依赖：进入 `01_可运行系统`，执行 `pip install -r requirements.txt`（首次需要联网，之后可完全离线运行）。
- 4. 双击 `光稳智检_iCAN演示启动.bat`（或执行 `python app.py`）。
- 5. 终端出现 **"系统启动完成！"** 与 访问地址：`http://localhost:5000` 后，浏览器打开 `http://127.0.0.1:5000`。
启动日志会打印 `已从本地加载 WideResNet-50 (IMAGENET1K_V2)`
`权重，无需联网，可据此确认处于离线状态。`
若评审不便克隆仓库，团队亦可按需另行提供打包副本；但源码与权重均以该仓库为准。

7.4.2 界面构成与交互说明

界面区域	交互方式	说明
当前运行模型	只读	固定为“光稳智检改进模型”；下方按钮打开离线对比数据
产品类别	下拉选择 + “加载模型”按钮	未加载时上传会被拒绝并给出提示；加载后显示当前类别与阈值
图像上传	点击“选择单张”或拖拽图片到虚线框	上传后进入预览状态，显示“清除 / 批量选择 / 开始检测”
批量选择	点击“批量选择”	选完文件即自动开始检测 ，无需再点“开始检测”；上传区有对应文字提示
检测设置	拖动阈值滑块	松开后 立即生效 （仅当前运行），可用于现场观察误报与召回的权衡；“重置阈值”恢复该类别默认值
可视化分析	点击“原图 / 热力图 / 叠加图”标签	批量检测后，上方出现逐张切换按钮，可单独查看批内任一张
检测历史	滚动查看；“导出报告”下载 CSV；“清空”二次确认后清空	历史保存在内存中， 重启服务后清空
检测统计	只读	会话累计值（含单张与批量），与“本批汇总”含义不同

7.4.3 标准操作流程

- 1. 选择类别（建议 `bottle` 或 `leather`）→ 点“加载模型”。
- 2. 上传正常样品 → 点“开始检测” → 结果应判为**正常**，分数低于阈值。
- 3. 上传缺陷样品 → 点“开始检测” → 结果应判为**异常**，热力图红色响应集中在缺陷位置。
- 4. 点“批量选择”并一次选中多张图 → 自动批量检测 → 查看本批汇总与逐张结果。
- 5. 在“可视化分析”中逐张切换，核对每张的热力图。
- 6. 点“导出报告”下载 CSV；需要时查看“离线实验对比”。

7.4.4 使用边界（必须说明）

- 演示版在本地 CPU 上运行，**不支持实时在基线与改进模型间切换**，也不支持未经训练的新类别一键上线。
- 热力图是局部分数相对阈值归一化后的可视化，**不是独立校准的缺陷概率**。
- 部分类别在当前阈值下误报率很高（如 `screw` 为 100%、`pill` 为 84.6%），不适合作为现场演示类别，详见 `01_可运行系统/README.md` 的类别选择建议。
- 公开演示不应替代人工质检，也不可直接用于生产放行。

八、目标用户与应用前景

8.1 应用领域

领域	应用场景	检测对象
电子制造	可探索的质量预筛	焊点、线路等外观异常
食品包装	可探索的包装外观预筛	印刷、封口等外观异常
纺织行业	可探索的面料预筛	污渍、破洞等外观异常

上述为潜在场景，并非已完成行业适配。目标用户是需要观察光照变化对视觉质检影响的教学团队、实验室和中小企业技术人员。实际部署仍需采集企业自有正常/缺陷样本，取得数据使用授权，并完成现场误报率、漏检率、吞吐量和稳定性验证。

8.2 技术优势

- 无监督学习：仅需正常样本
- 本地演示：具备类别选择、批量检测、热力图及报告导出
- 灵活配置：支持多类别、可调阈值

8.3 推广与商业化路径

先以教学和实验室演示收集反馈，再与有明确检测对象的企业开展小规模试点；只有在现场验证达标后，才考虑按部署、维护或定制服务收费。公开基准数据用于研究评测，不能直接作为可商业化训练资产；商业化应使用权属清晰的企业数据与合法授权模型。

九、团队介绍

9.1 指导教师

姓名	职称	单位	研究方向
贾子琪	讲师	南阳理工学院 计算机与软件学院 人工智能教研室	人工智能/计算机科学与技术
郭东恩	教授（副院长）	南阳理工学院 计算机与软件学院	人工智能/计算机科学与技术

9.2 团队成员

角色	姓名	学校	分工
项目负责人	申浩远	南阳理工学院	算法设计、模型开发、系统集成
成员1	邱海霞	郑州轻工业大学	数据整理、光照增强
成员2	陈世钊	南阳理工学院	评测与报告
成员3	韩泷辉	南阳理工学院	原型开发、界面设计

以上成员的报名身份、学号与年级，以团队在赛事报名系统中提交的报名信息为准。

十、项目规划

10.1 开发计划与当前进度

阶段	时间	主要任务	状态
需求调研与方案设计	2026.08	调研工业质检误报痛点、确定光照鲁棒技术路线、搭建可运行原型	已完成
算法实现与实验	2026.09 上旬	PatchCore 基线复现、光照鲁棒增强实现，完成 MVTec AD 15 类与 MVTec AD 2 8 类实验	已完成
系统开发与联调	2026.09 中旬	Flask 前后端开发、批量检测/热力图/报告导出、离线依赖本地化	已完成
指标复核与材料整理	2026.09 下旬	阈值校准复核、全包数据口径一致性核对、方案书与演示视频制作	进行中
现场数据验证与产品化评估	2026.10–2026.12	采集真实产线光照数据、验证误报率与节拍、评估工程化落地路径	计划中

10.2 未来规划

- 短期：完善系统功能，补充更多类别支持
- 中期：与工业企业合作试点，验证实际效果
- 长期：在完成现场验证与数据授权后，评估产品化推广

附录

参考文献

1. Roth, K., et al. "Towards Total Recall in Industrial Anomaly Detection." CVPR 2022.

2. Bergmann, P., et al. "MVTec AD – A Comprehensive Real-World Dataset." CVPR 2019.

3. Defard, T., et al. "PaDiM: a Patch Distribution Modeling Framework." ICPR 2021.

4. Bergmann, P., et al. "The MVTec AD 2 Dataset: Advanced Scenarios for Unsupervised Anomaly Detection." IJCV 2025.

联系方式

- 学校：南阳理工学院 计算机与软件学院
- 指导教师：贾子琪（讲师）、郭东恩（教授/副院长）
- 邮箱：924362481@qq.com

文档版本： V3.1（提交终审版） **修订日期：** 2026年9月27日